

CAN–Ethernet Bridge IP Core

Functional Specification

Bidirectional CAN 2.0B $\leftrightarrow$ 100 Mbps Ethernet Gateway/Tunnel —
Interface Definitions, Functional Requirements, Register Map, Protocol
Specification

IIITDM Kancheepuram · Final Year Project · Rev 1.0 · May 2026

Contents

1	1. Scope and Purpose	2
2	2. System Overview	2
3	3. Top-Level Interface	2
3.1	3.1 Clock and Reset	2
3.2	3.2 CAN Wishbone Master (8-bit)	2
3.3	3.3 ETH Wishbone Master (32-bit)	3
3.4	3.4 Host Wishbone Slave (32-bit)	3
3.5	3.5 Interrupts	3
4	4. Register Map	4
5	5. Encapsulation Protocol	4
5.1	5.1 Frame Format (Word-level, 32-bit aligned)	4
5.2	5.2 Downstream Validation Rules	5
6	6. Functional Requirements	5
6.1	6.1 Operating Modes	5
6.2	6.2 Upstream Path (CAN $\rightarrow$ ETH)	6
6.3	6.3 Downstream Path (ETH $\rightarrow$ CAN)	6
6.4	6.4 CAN ID Filter	6
6.5	6.5 Bus Arbitration	7
6.6	6.6 Reset Behaviour	7
7	7. Timing Requirements	7
8	8. Error Handling	8
9	9. Revision History	8

1. Scope and Purpose

This document specifies the functional behaviour of the CAN-Ethernet Bridge IP core. It covers interface definitions, the custom encapsulation protocol, register specifications, functional requirements, and timing constraints.

The bridge enables transparent, bidirectional data transfer between a CAN 2.0B bus (1 Mbps) and a 100 Mbps Ethernet network. It operates as a Wishbone-based IP core with three bus interfaces: an 8-bit WB master to the CAN controller, a 32-bit WB master to the Ethernet MAC/RAM, and a 32-bit WB slave for host CPU configuration.

2. System Overview

Parameter	Value	Notes
CAN interface	8-bit Wishbone master	SJA1000-compatible register set
ETH interface	32-bit Wishbone master	OpenCores Ethernet MAC + shared RAM
Host interface	32-bit Wishbone slave	32 registers (0x00–0x7C)
Operating modes	Gateway / Tunnel	Software-selectable
CAN ID filter	16 entries, 29-bit	Accept-list or reject-list
CAN TX queue	8-deep FIFO	Absorbs ETH→CAN speed gap
Clock domain	Single (50 MHz)	No internal CDC
Reset	Active-low synchronous	Hybrid defaults (CPU-less boot)

3. Top-Level Interface

Module: `can_eth_bridge_top` (`rtl/can_eth_bridge_top.v`)

3.1 Clock and Reset

Port	Dir	Width	Description
<code>clk</code>	in	1	System clock. All registers are positive-edge triggered. Nominal 50 MHz.
<code>rst_n</code>	in	1	Active-low synchronous reset. All FSMs return to IDLE; config registers load hybrid defaults.

3.2 CAN Wishbone Master (8-bit)

Port	Dir	Width	Description
can_wbm_adr_o	out	8	CAN controller register address
can_wbm_dat_o	out	8	Write data to CAN controller
can_wbm_dat_i	in	8	Read data from CAN controller
can_wbm_cyc_o	out	1	Bus cycle active
can_wbm_stb_o	out	1	Valid transfer strobe
can_wbm_we_o	out	1	Write enable
can_wbm_ack_i	in	1	Slave acknowledge

3.3 ETH Wishbone Master (32-bit)

Port	Dir	Width	Description
eth_wbm_adr_o	out	32	Memory / BD address
eth_wbm_dat_o	out	32	Write data
eth_wbm_dat_i	in	32	Read data
eth_wbm_sel_o	out	4	Byte lane select
eth_wbm_cyc_o	out	1	Bus cycle active
eth_wbm_stb_o	out	1	Valid transfer strobe
eth_wbm_we_o	out	1	Write enable
eth_wbm_ack_i	in	1	Slave acknowledge

3.4 Host Wishbone Slave (32-bit)

Port	Dir	Width	Description
host_wb_adr_i	in	8	Register address (0x00–0x7C)
host_wb_dat_i	in	32	Write data from host
host_wb_dat_o	out	32	Read data to host
host_wb_cyc_i	in	1	Bus cycle
host_wb_stb_i	in	1	Strobe
host_wb_we_i	in	1	Write enable
host_wb_ack_o	out	1	Acknowledge

3.5 Interrupts

Port	Dir	Width	Description
can_irq_i	in	1	CAN frame received (triggers upstream FSM)
eth_tx_irq_i	in	1	Ethernet TX complete (upstream release)
eth_rx_irq_i	in	1	Ethernet frame received (triggers downstream FSM)

Port	Dir	Width	Description
can_tx_irq_i	in	1	CAN TX complete (downstream free BD)

4. Register Map

All registers are 32-bit, accessed at 4-byte aligned offsets via the Host WB slave.

Offset	Register	Access	Reset	Bit-fields
0x00	BRIDGE_CTRL	R/W	0x00000001	[0] bridge_en, [2:1] mode (01=GW, 10=TUN)
0x04	BRIDGE_STATUS	RO	0x00000000	[0] up_busy, [1] dn_busy, [7:4] queue_depth
0x08	DST_MAC_HI	R/W	0xFFFFFFFF	Destination MAC [47:16]
0x0C	DST_MAC_LO	R/W	0x0000FFFF	Destination MAC [15:0]
0x10	SRC_MAC_HI	R/W	0x02CAFE00	Source MAC [47:16]
0x14	SRC_MAC_LO	R/W	0x00000001	Source MAC [15:0]
0x18	FILTER_CTRL	R/W	0x00000000	[0] filter_en, [1] filter_mode
0x1C–0x58	FILTER_TBL[0:15]	R/W	0x00000000	16 × 29-bit CAN ID filter entries
0x5C	CNT_CAN_RX	RO	0x00000000	CAN frames received
0x60	CNT_ETH_TX	RO	0x00000000	Ethernet frames transmitted
0x64	CNT_ETH_RX	RO	0x00000000	Ethernet frames received
0x68	CNT_CAN_TX	RO	0x00000000	CAN frames transmitted
0x6C	CNT_FILTERED	RO	0x00000000	Frames dropped by filter
0x70	CNT_ERRORS	RO	0x00000000	Validation errors
0x74	TUNNEL_SEQ	RO	0x00000000	Tunnel sequence counter
0x78	TX_BD_ADDR	R/W	0x00000400	ETH TX buffer descriptor address
0x7C	RX_BD_ADDR	R/W	0x00000600	ETH RX buffer descriptor address

5. Encapsulation Protocol

5.1 Frame Format (Word-level, 32-bit aligned)

Word	Bits	Field
0	[31:0]	DST MAC [47:16]
1	[31:16]	DST MAC [15:0]
1	[15:0]	SRC MAC [47:32]
2	[31:0]	SRC MAC [31:0]
3	[31:16]	EtherType (0xCAFE gateway, 0xCABE tunnel)

Word	Bits	Field
3	[15:8]	Magic high byte (0xB1)
3	[7:0]	Magic low byte (0xDE)
4	[31:24]	Protocol version (0x01)
4	[23:0]	CAN ID upper bits
5	[31:28]	CAN ID lower / EFF / RTR flags
5	[27:24]	DLC (0–8)
5	[23:16]	Flags (mode, filter status)
5	[15:0]	Reserved
6	[31:16]	Tunnel sequence number (tunnel mode only)
6	[15:0]	Timestamp (20 μ s resolution)
7–8	[31:0]	CAN payload data (up to 8 bytes, zero-padded)
9–15	[31:0]	Padding to 64-byte Ethernet minimum

5.2 Downstream Validation Rules

A received Ethernet frame is accepted if **all** of the following are true:

1. EtherType matches the current bridge mode (0xCAFE for gateway, 0xCABE for tunnel)
2. Magic bytes = 0xB1DE
3. Protocol version = 0x01
4. $\text{DLC} \leq 8$

If any check fails, the frame is dropped and `cnt_errors` is incremented.

6. Functional Requirements

6.1 Operating Modes

ID	Requirement
FR-01	The bridge shall support gateway mode (EtherType 0xCAFE).
FR-02	The bridge shall support tunnel mode (EtherType 0xCABE) with auto-incrementing sequence numbers and timestamps.
FR-03	The bridge shall be software-disableable via bridge_en bit; when disabled, no frames shall be forwarded.
FR-04	Mode switching shall take effect within 1 clock cycle of register write.

6.2 Upstream Path (CAN → ETH)

ID	Requirement
FR-10	Upon CAN RX interrupt, the bridge shall read 13 bytes from the CAN controller (frame info + up to 8 data bytes).
FR-11	Standard (11-bit) and Extended (29-bit, EFF) CAN IDs shall be supported.
FR-12	DLC values 0–8 shall be accepted; DLC > 8 shall be treated as 8.
FR-13	If the CAN ID filter is enabled, frames matching the filter criteria shall be dropped before encapsulation.
FR-14	The encapsulated Ethernet frame shall be written to shared RAM (16 × 32-bit words).
FR-15	After RAM write, the bridge shall program the TX Buffer Descriptor with length and READY bit.
FR-16	The bridge shall wait for ETH TX complete interrupt before releasing the buffer.
FR-17	Counter <code>cnt_can_rx</code> shall increment on each successful CAN read; <code>cnt_eth_tx</code> on each ETH transmit.

6.3 Downstream Path (ETH → CAN)

ID	Requirement
FR-20	Upon ETH RX interrupt, the bridge shall read the RX Buffer Descriptor (2 words).
FR-21	The bridge shall read the Ethernet frame from RAM (16 × 32-bit words).
FR-22	The frame shall be validated: EtherType, magic bytes (0xB1DE), protocol version (0x01), and DLC ≤ 8.
FR-23	Invalid frames shall be dropped; <code>cnt_errors</code> shall increment.
FR-24	Valid frames shall be decapsulated and the CAN frame enqueued in the TX queue.
FR-25	If the TX queue is full (8 entries), the FSM shall stall until space is available.
FR-26	The bridge shall write 13 bytes to the CAN controller and issue a TX request.
FR-27	The bridge shall wait for CAN TX complete interrupt, then free the RX BD.

6.4 CAN ID Filter

ID	Requirement
FR-30	The filter shall support 16 entries, each holding a 29-bit CAN ID.
FR-31	Accept-list mode (<code>filter_mode=0</code>): only IDs present in the table are forwarded.
FR-32	Reject-list mode (<code>filter_mode=1</code>): IDs present in the table are dropped.
FR-33	When <code>filter_en=0</code> , all frames shall pass (no filtering).
FR-34	<code>cnt_filtered</code> shall increment for each dropped frame.

6.5 Bus Arbitration

ID	Requirement
FR-40	The CAN bus and ETH bus shall each have a round-robin arbiter.
FR-41	Both upstream and downstream FSMs may request either bus simultaneously.
FR-42	No grant overlap: only one master shall be granted at any time per bus.
FR-43	No starvation: alternating priority ensures both FSMs are served.

6.6 Reset Behaviour

ID	Requirement
FR-50	On assertion of <code>rst_n=0</code> , all FSMs shall return to IDLE within 1 clock.
FR-51	No partial Buffer Descriptors shall be left in READY state after reset.
FR-52	All counters shall be cleared to zero on reset.
FR-53	Configuration registers shall load hybrid defaults (bridge enabled, gateway mode, broadcast DST MAC).

7. Timing Requirements

Parameter	Value	Notes
Target clock frequency	50 MHz	Single domain, no CDC
Upstream latency	$\sim 8.75 \mu\text{s}$	CAN RX IRQ $\rightarrow$ BD READY
Downstream latency	$\sim 232 \mu\text{s}$	ETH RX IRQ $\rightarrow$ CAN TX request
Timestamp resolution	$20 \mu\text{s}$	50 MHz / 1000 prescaler

Parameter	Value	Notes
CAN WB transfer	1 cycle/byte	8-bit bus, single-cycle ACK
ETH WB transfer	1 cycle/word	32-bit bus, single-cycle ACK
Max upstream throughput	~114 kfps	CAN frames per second
Max downstream throughput	~4.3 kfps	Bottleneck: 13-byte CAN writes

8. Error Handling

Error Condition	Bridge Response
Invalid EtherType	Frame dropped, <code>cnt_errors</code> incremented
Invalid magic ($\neq 0xB1DE$)	Frame dropped, <code>cnt_errors</code> incremented
Invalid version ($\neq 0x01$)	Frame dropped, <code>cnt_errors</code> incremented
DLC > 8	Frame dropped, <code>cnt_errors</code> incremented
CAN TX queue full	FSM stalls at <code>QUEUE_CHK</code> until space available
Mid-transaction reset	FSMs return to IDLE, no partial BD left READY

9. Revision History

Rev	Date	Author	Changes
1.0	May 2026	FYP / IIITDM	Initial release. Covers gateway/tunnel modes, full register map, encapsulation protocol, 28 functional requirements.